

Ecological forecasting in R

Lecture 4: evaluating dynamic models

Nicholas Clark

School of Veterinary Science, University of Queensland

0900–1200 CET Wednesday 26th March, 2025

Workflow

Press the "o" key on your keyboard to navigate among slides

Access the [tutorial html here](#)

Download the data objects and exercise  script from the html file

Complete exercises and use Slack to ask questions

Relevant open-source materials include:

[Evaluating distributional forecasts](#)

[Approximate leave-future-out cross-validation for Bayesian time series models](#)

[The Marginal Effects Zoo \(0.14.0\)](#)

This lecture's topics

Forecasting from dynamic models

Prediction-based inferences

Bayesian posterior predictive checks

Probabilistic forecast evaluation

Forecasting from dynamic models

Forecasting in `mvgam`

Two options

Pass `newdata` into the `mvgam()` function for automatic probabilistic forecasts through `Stan`

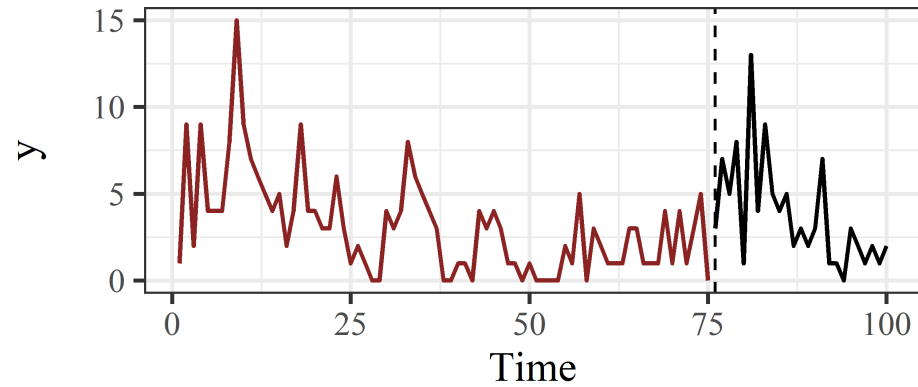
Produce forecasts outside of `Stan` by passing `newdata` and the fitted model to the `forecast()` function

Both require any out-of-sample covariates to be supplied

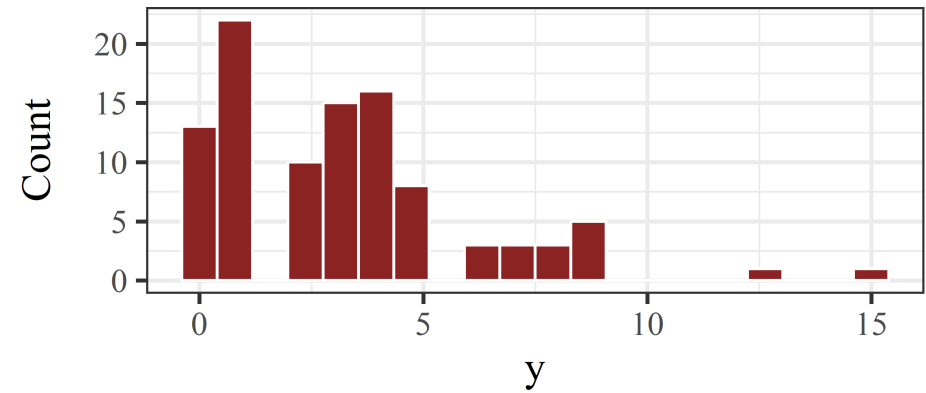
Both should give equivalent results

Simulated data

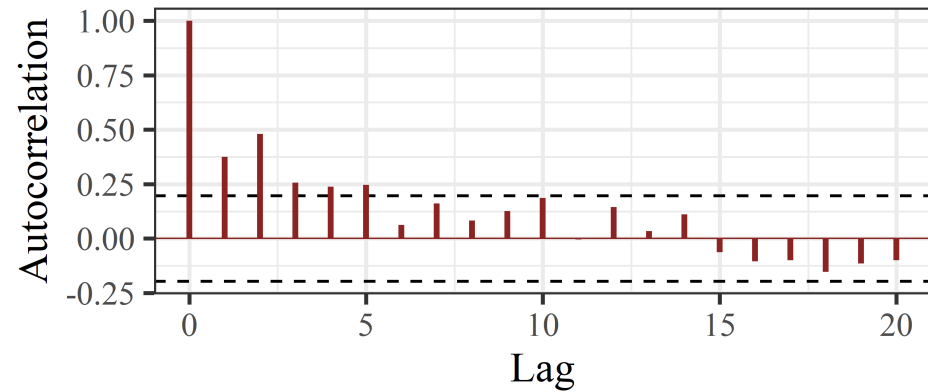
Time series



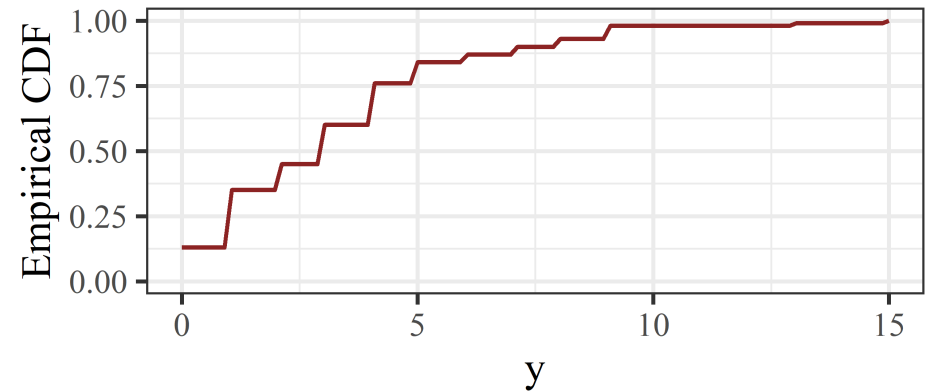
Histogram



ACF



CDF



The model

```
library(mvgam)
model <- mvgam(
  y ~
    s(season, bs = 'cc', k = 8),
  data = data_train,
  newdata = data_test,
  trend_model = GP(),
  family = poisson()
)
```

A cyclic smooth of `season` to capture repeated periodic variation

The model

```
library(mvgam)
model <- mvgam(
  y ~
    s(season, bs = 'cc', k = 8),
  data = data_train,
  newdata = data_test,
  trend_model = GP(),
  family = poisson()
)
```

A Gaussian Process trend (approximated with Hilbert basis functions)

The model

```
library(mvgam)
model ← mvgam(
  y ~
    s(season, bs = 'cc', k = 8),
  data = data_train,
  newdata = data_test,
  trend_model = GP(),
  family = poisson()
)
```

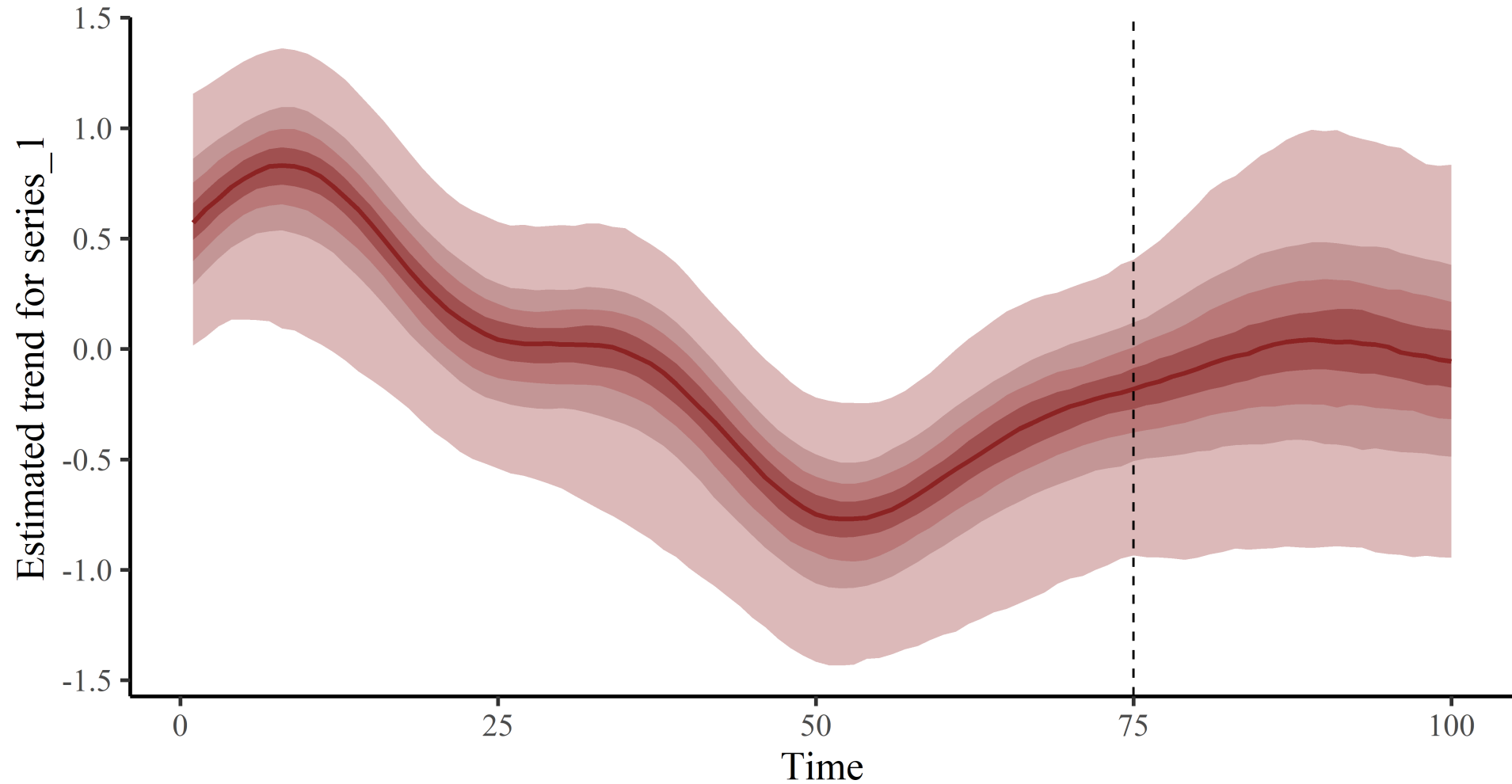
Forecasts computed automatically using the generated quantities
block in Stan

Dropping newdata

```
model2 <- mvgam(  
  y ~  
    s(season, bs = 'cc', k = 8),  
  data = data_train,  
  trend_model = GP(),  
  family = poisson()  
)
```

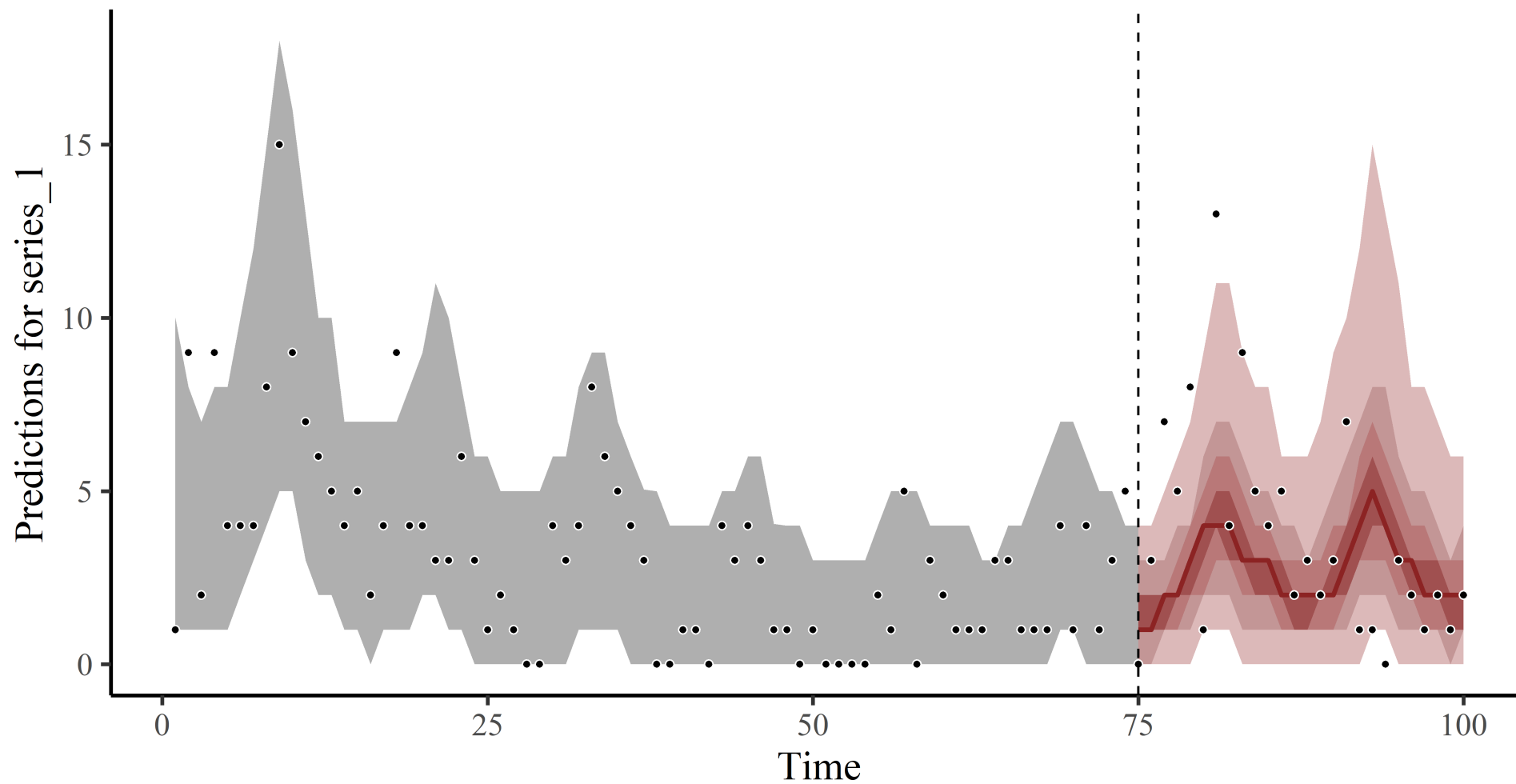
Predictions will only be calculated for the training data if no testing data (i.e. `newdata`) are supplied

```
plot(model, type = 'trend', newdata = data_test)
```



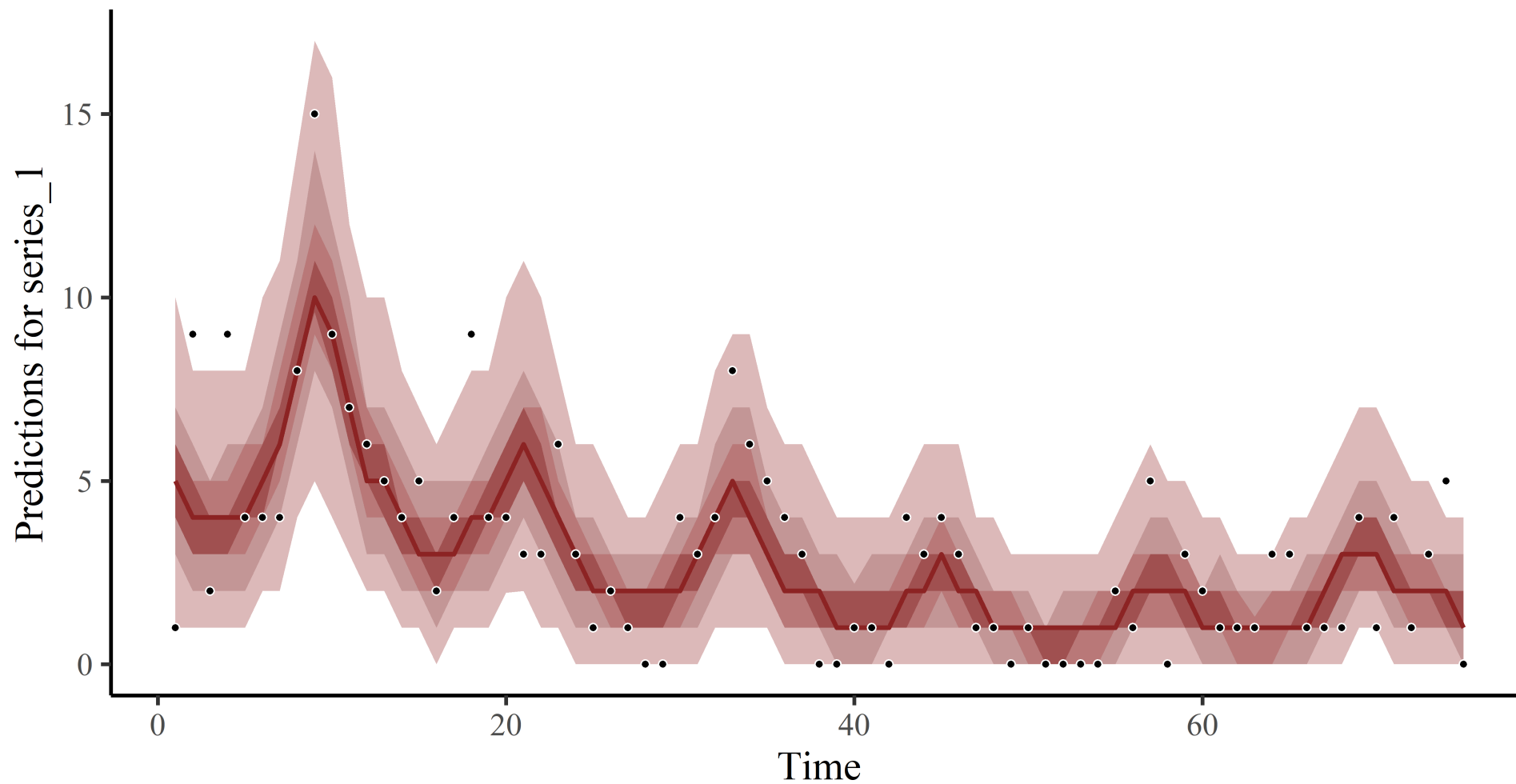
Trend extends into the future

```
plot(forecast(model))
```



Forecasts can be compared to truths quickly

```
plot(hindcast(model2))
```



No forecasts available. Now what?

Posterior draws

Dynamic `mvgam` models contain draws for many quantities

- β coefficients for linear predictor terms (called `b`)

- Any family-specific shape / scale parameters (i.e. ϕ for Negative Binomial; σ_{obs} for Normal / LogNormal etc...)

- Any trend-specific parameters (i.e. α and ρ for GP trends; σ and $ar1$ for AR trends etc...)

- In-sample posterior predictions (called `ypred`)

- In-sample posterior trend estimates (called `trend`)

All stored as MCMC draws in an object of class `stanfit` in the `model_output` slot

The stanfit object

```
summary(model2$model_output)
```

```
## $summary
##              mean      se_mean      sd      2.5%      25%
## alpha_gp[1]    0.62171563 0.006264558 0.1927977 3.259427e-01 0.4759840000
## rho_gp[1]      12.69452436 0.472567665 9.0984607  2.409932e+00 6.6534600000
## b_gp[1,1]      -0.08784042 0.028832518 0.9571218 -1.970965e+00 -0.7333785000
## b_gp[2,1]       1.03580834 0.018317140 0.7738592 -4.485449e-01 0.5047912500
## b_gp[3,1]       0.55880067 0.021205927 0.8509277 -1.161346e+00 -0.0172447250
## b_gp[4,1]      -1.22545951 0.021101693 0.8203285 -2.905063e+00 -1.7379950000
## b_gp[5,1]      -0.68120807 0.019720616 0.7682425 -2.147765e+00 -1.2233075000
## b_gp[6,1]       0.58285264 0.021051423 0.8264262 -9.898045e-01 0.0222727500
## b_gp[7,1]       0.23068307 0.019710134 0.8508540 -1.432141e+00 -0.3553967500
## b_gp[8,1]       0.11852579 0.021177339 0.8353253 -1.565237e+00 -0.4458395000
## b_gp[9,1]       0.33095648 0.019470049 0.8352777 -1.283770e+00 -0.2210705000
## b_gp[10,1]     -0.17751874 0.020550968 0.8664645 -1.799261e+00 -0.7721652500
## b_gp[11,1]     -0.58467981 0.022633968 0.9228061 -2.328848e+00 -1.2267625000
## b_gp[12,1]     -0.02275813 0.019498238 0.8754558 -1.769344e+00 -0.5927065000
## b_gp[13,1]      0.49333417 0.025280945 0.9395104 -1.480099e+00 -0.1008087500
```

Draws of trend

Code Plot

```
# view posterior draws of the trend  
plot(model2, type = 'trend', realisations = TRUE,  
      n_realisations = 10)
```


But how can we extrapolate these to the future?

Ready for some multivariate statistical wizardry?

Ready





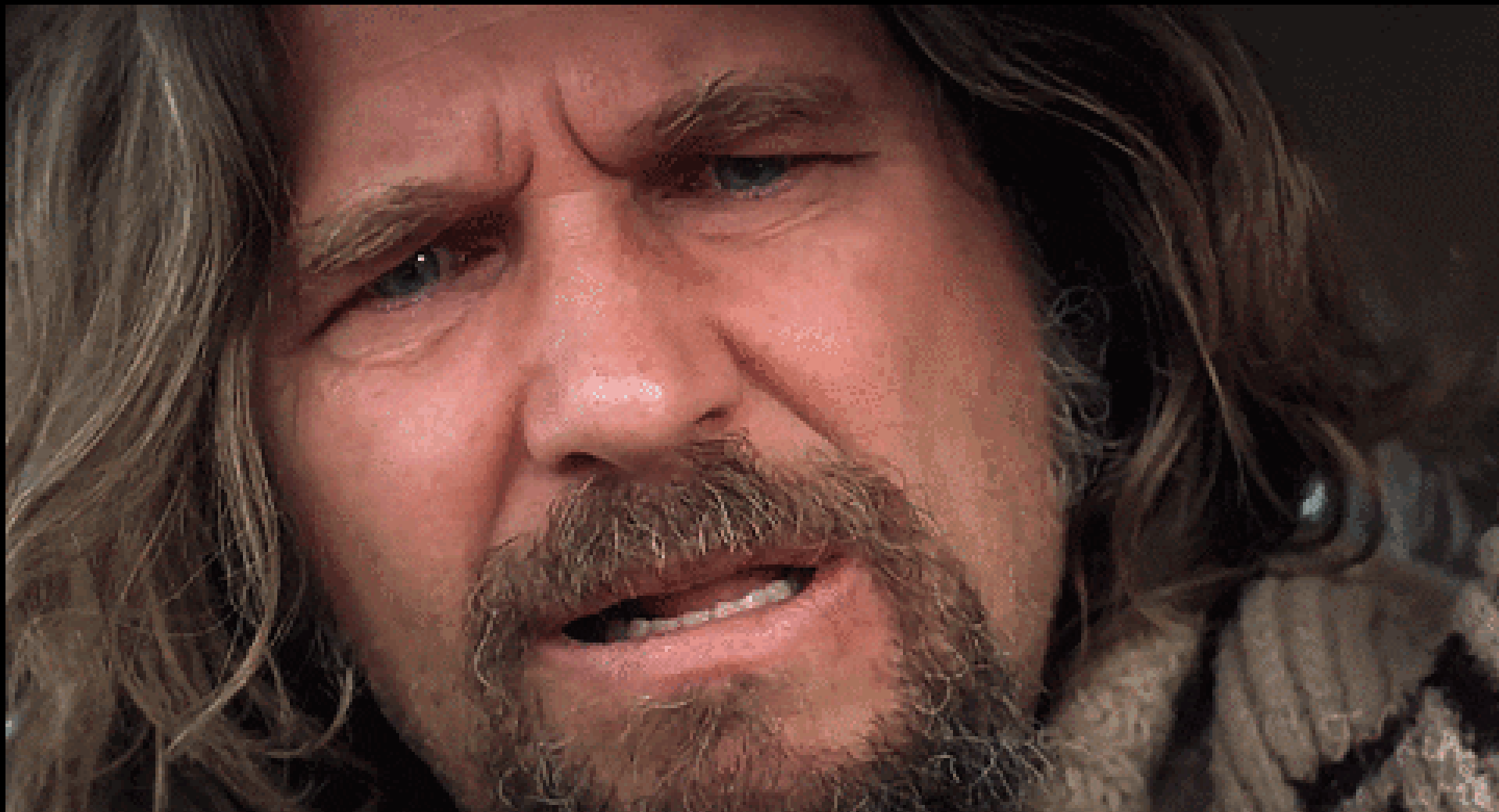
```
sim_gp = function(trend_draw, h, rho, alpha){  
  # extract training and testing times  
  t ← 1:length(trend_draw); t_new ← 1:(length(trend_draw) + h)  
  # calculate training covariance  
  Sigma ← alpha ^ 2 * exp(-0.5 * ((outer(t, t, "-") / rho) ^ 2)) +  
    diag(1e-9, length(t))  
  # calculate training vs testing cross-covariance  
  Sigma_new ← alpha ^ 2 * exp(-0.5 * ((outer(t, t_new, "-") / rho) ^ 2))  
  # calculate testing covariance  
  Sigma_star ← alpha ^ 2 * exp(-0.5 * ((outer(t_new, t_new, "-") / rho) ^ 2))  
+  
    diag(1e-9, length(t_new))  
  # draw one function realization of the stochastic Gaussian Process  
  t(Sigma_new) %*% solve(Sigma, trend_draw) +  
    MASS::mvrnorm(1, mu = rep(0, length(t_new)),  
      Sigma = Sigma_star - t(Sigma_new) %*% solve(Sigma,  
Sigma_new))  
}
```

Wizardize one trend draw

Wizardry Plot

```
# extract trend parameter draws and plot one draw
trend_draws <- as.matrix(model2, variable = 'trend', regex = TRUE)
alpha_draws <- as.matrix(model2, variable = 'alpha_gp', regex = TRUE)
rho_draws <- as.matrix(model2, variable = 'rho_gp', regex = TRUE)
plot(1, type = 'n', bty = 'l',
     xlim = c(1, 130), ylim = range(trend_draws[1,]),
     ylab = 'One trend draw', xlab = 'Time')
lines(trend_draws[1,], col = 'gray70', lwd = 3.5)
# wizardize to extend draw forward 30 timesteps and plot
forecast_draw = sim_gp(trend_draw = trend_draws[1,], h = 30
                      alpha = alpha_draws[1,], rho = rho_draws[1,])
lines(x = 101:130, y = forecast_draw[101:130], lwd = 3.5, col = 'darkred')
abline(v = 100.5, lty = 'dashed', lwd = 2.5)
```

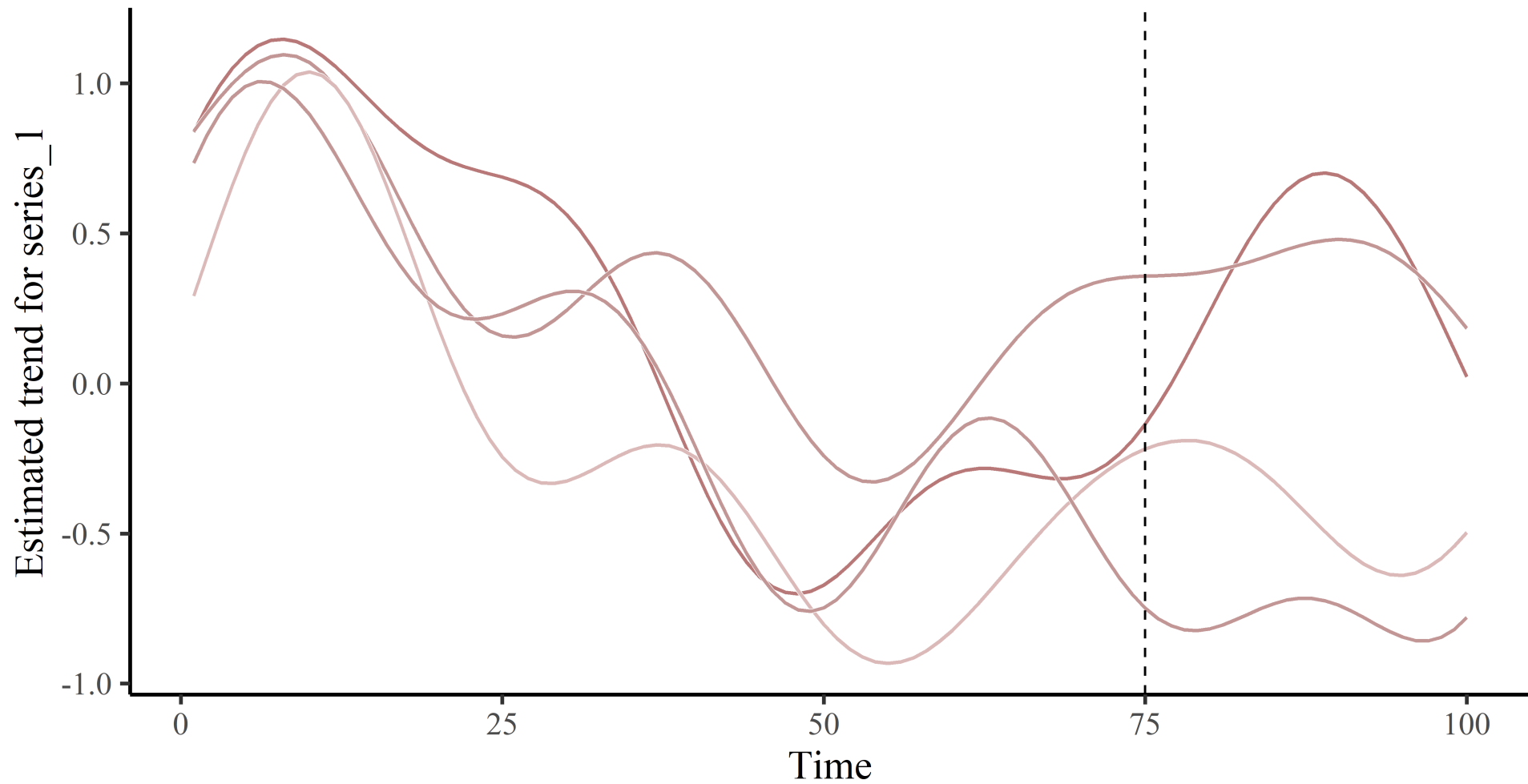
Piece of cake?



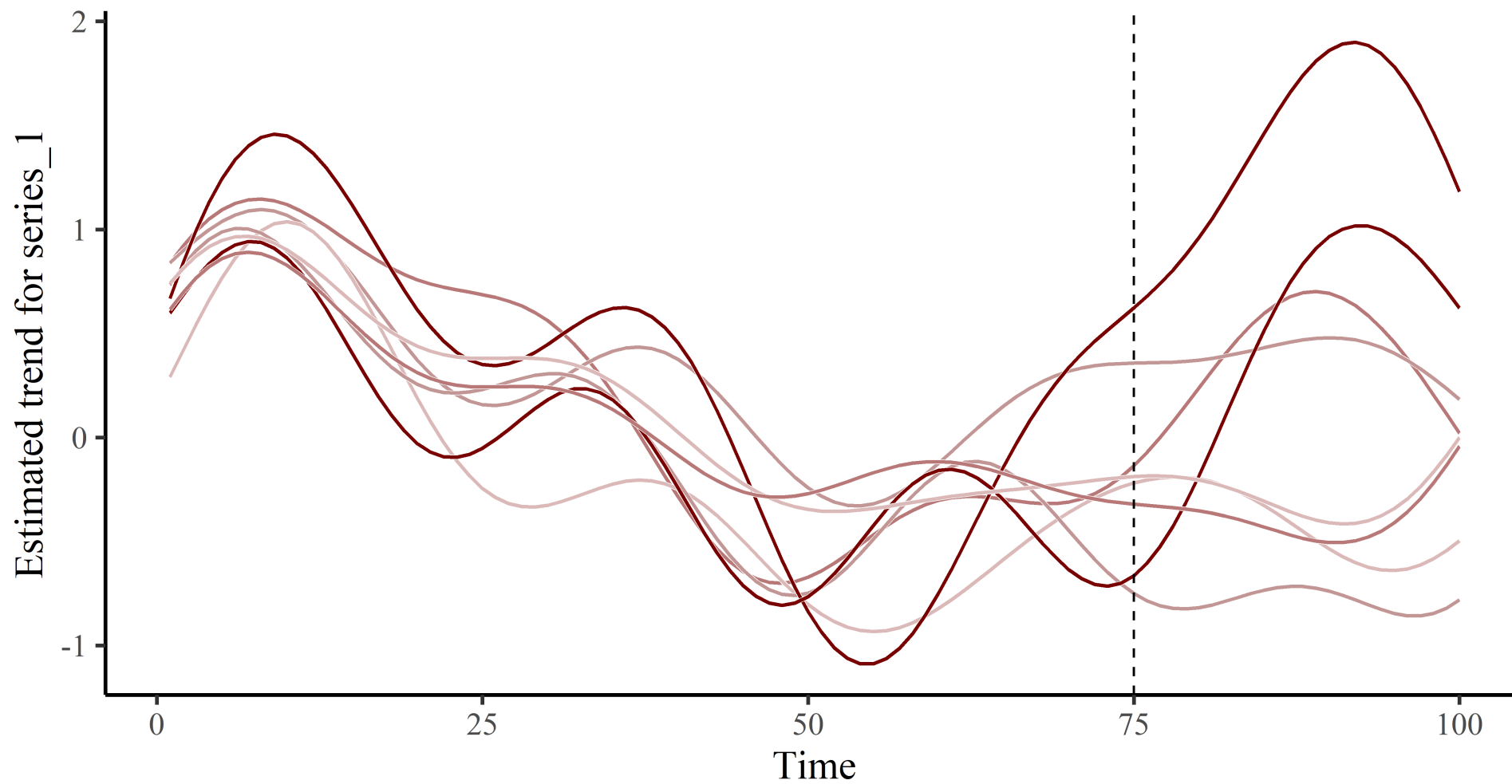
There is no wizardry ☹️. Rather, each kind of trend (AR, GP etc...) has an underlying stochastic equation that can be used to extrapolate draws to the future

But doing this manually is slow and error-prone. `mvgam` does this *automatically* using `newdata`

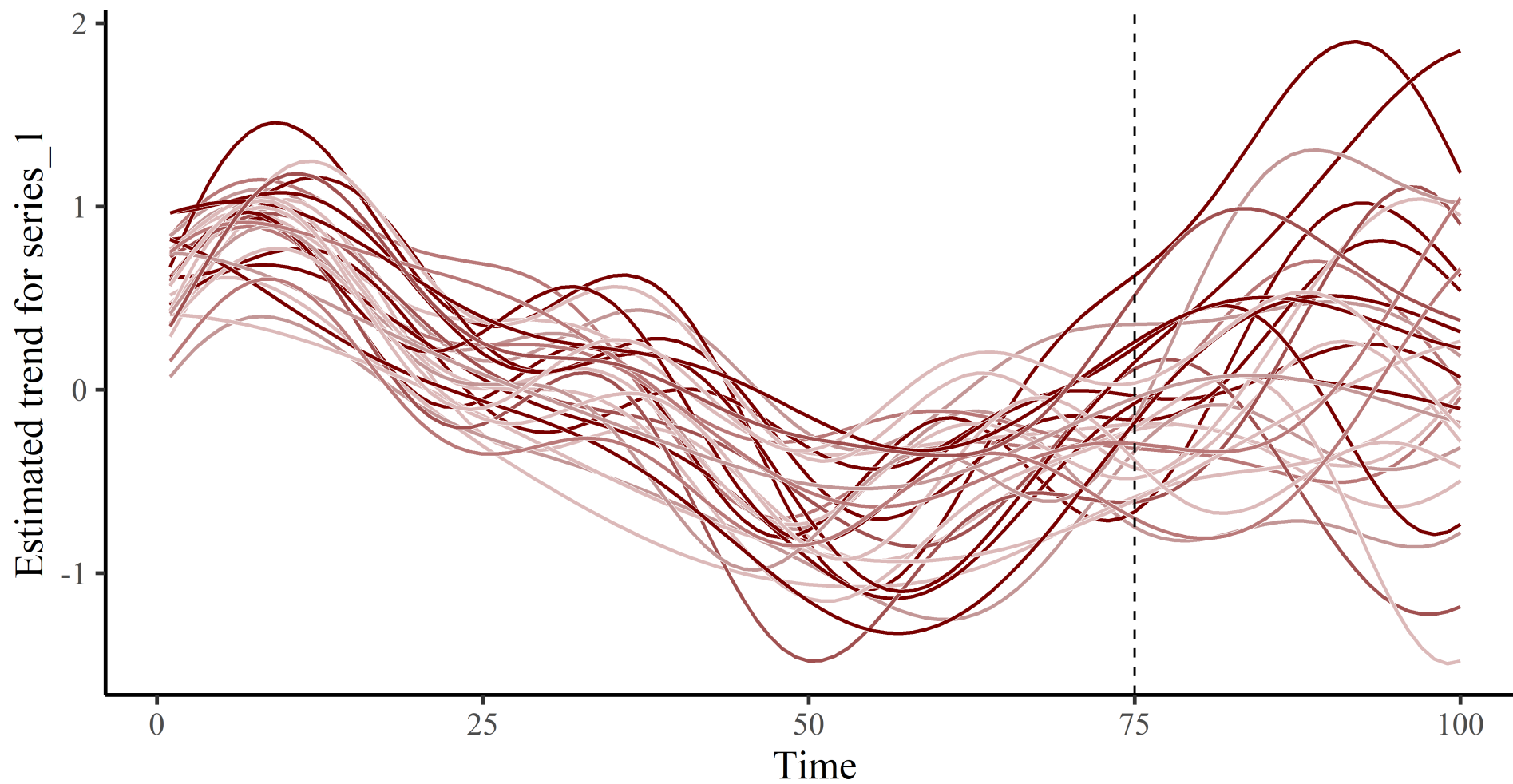
```
plot(model2, type = 'trend', n_realisations = 4)
```



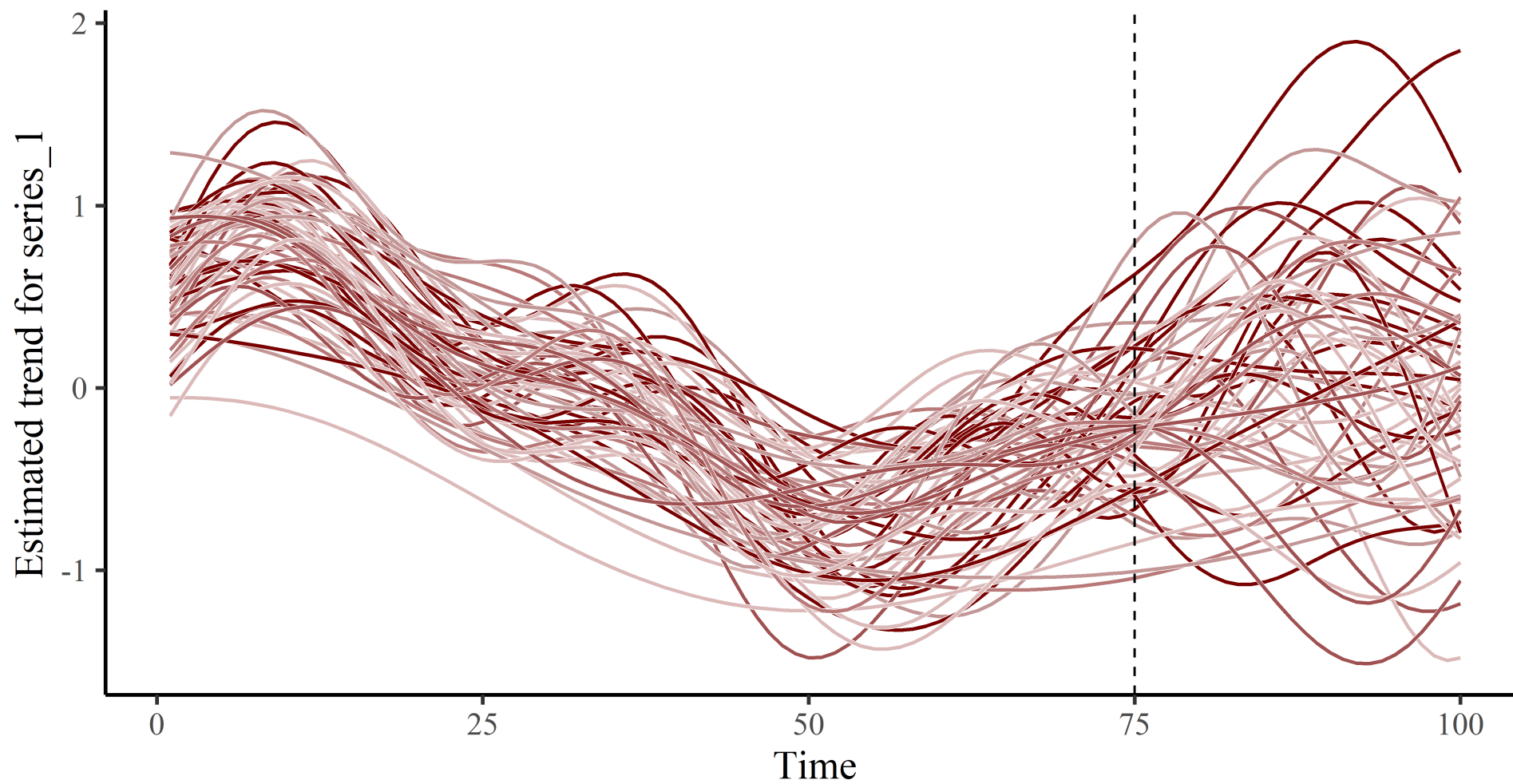
```
plot(model2, type = 'trend', n_realisations = 8)
```



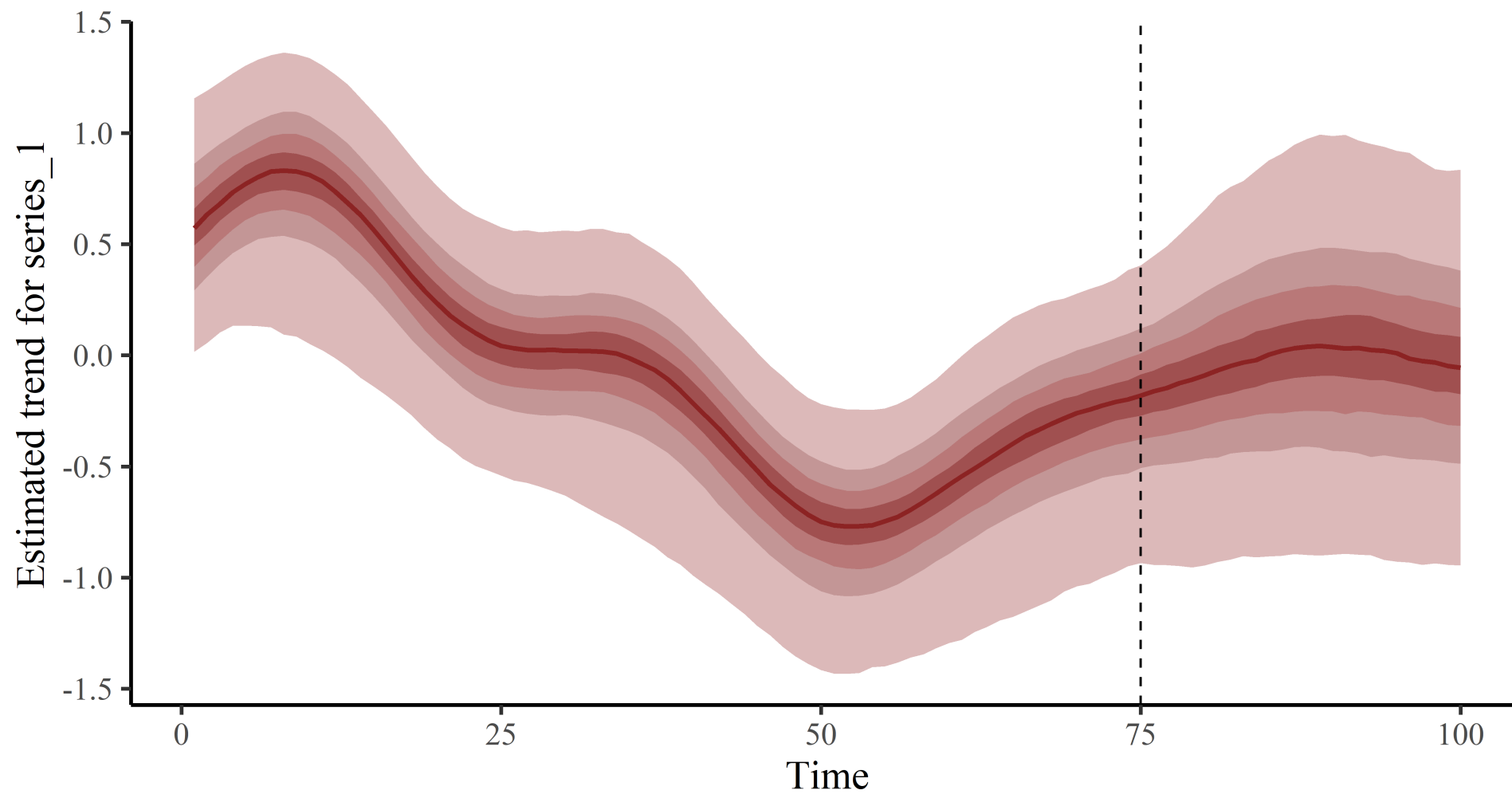

```
plot(model2, type = 'trend', n_realisations = 30)
```



```
plot(model2, type = 'trend', n_realisations = 60)
```



```
plot(model2, type = 'trend')
```



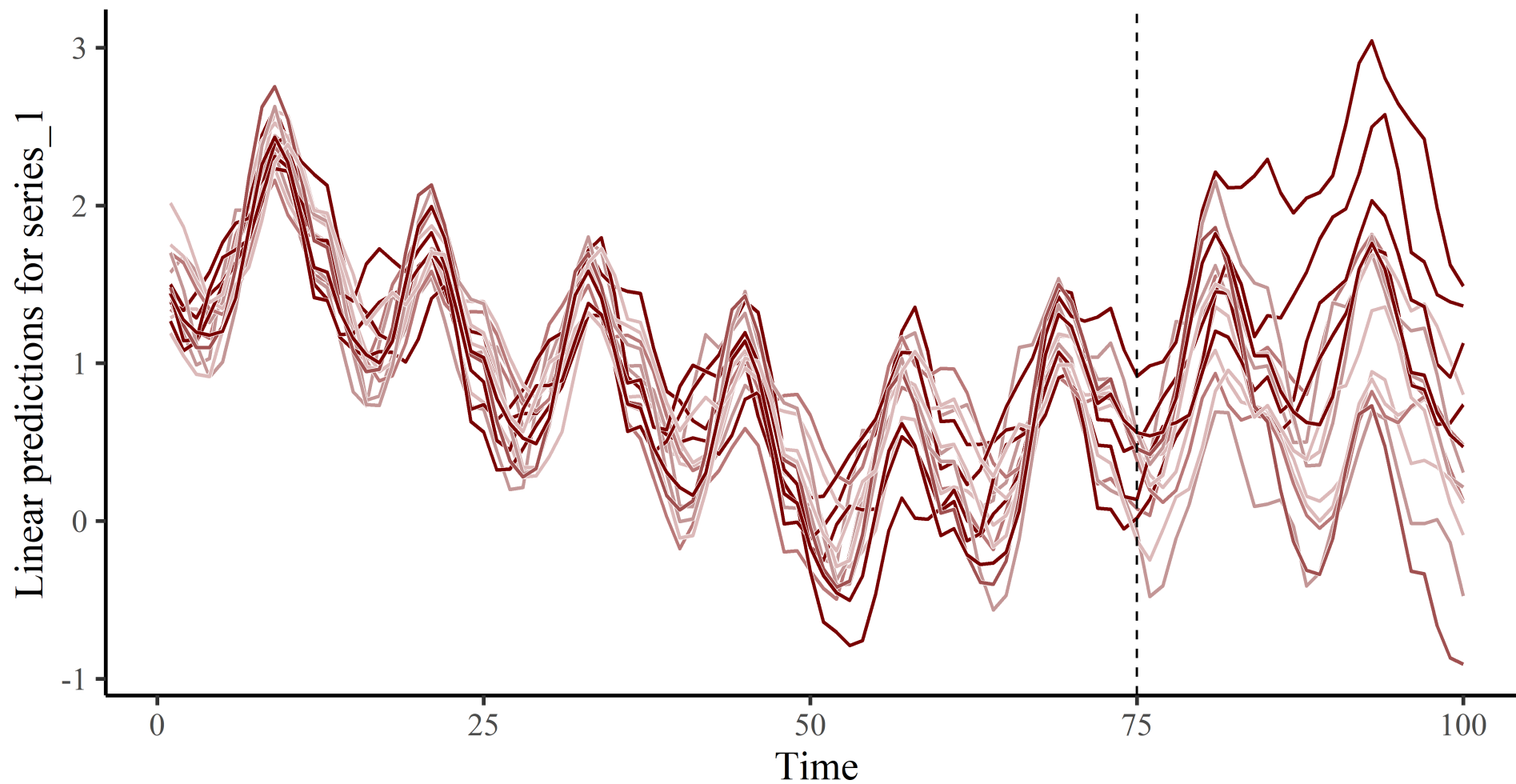
Once dynamic trend is extrapolated, computing forecasts is easy

**We only need to supply any remaining "future" predictor values
from covariates**

Covariate predictions are added to the trend predictions to give the full predictions *on the link scale*

`mvgam` does this *automatically* using the `forecast()` function

```
plot(forecast(model2, type = 'link'), realisations = TRUE)
```



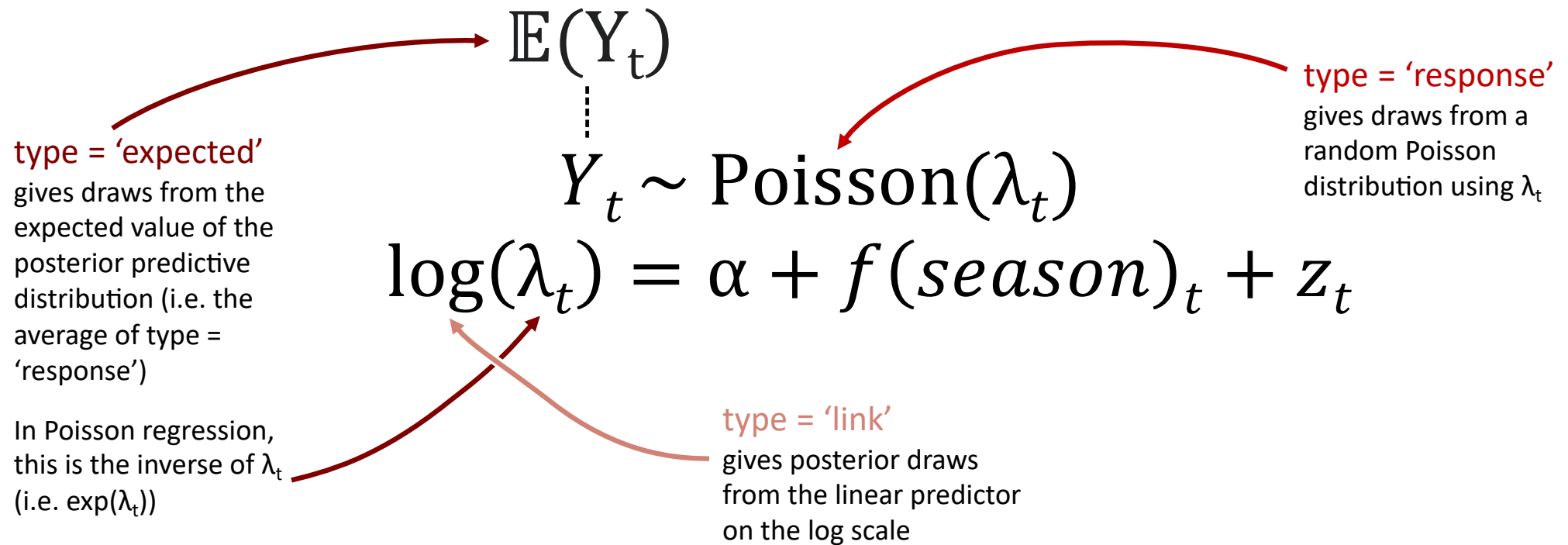
Live code example

Forecasting is easier if `newdata` passed to `mvgam()`, but this results in a larger model object and requires test data be available now

When testing data not available, you can generate forecasts for new data later using `forecast()` (note, `time` values in `newdata` must follow immediately from `time` values in original training data)

But there are multiple *types* of predictions available. What are they?

Types of `mvgam` predictions



modified from [Heiss 2022](#)

mvgam and brms 's

Type	mvgam	brms
link	<code>predict(type = 'link')</code>	<code>posterior_linpred()</code>
expected	<code>predict(type = 'expected')</code>	<code>posterior_epred()</code>
response	<code>predict(type = 'response')</code>	<code>posterior_predict()</code>

For all `mvgam` predictions, whether to include error in the dynamic process can be controlled using `process_error = TRUE` or `process_error = FALSE`

How should *you* use these predictions?

1. **Don't interpret coefficients. Rather, use predictions on both the link and expectation scale to understand the model better**
2. **Compare predictions on the response scale to the observed data using posterior predictive checks and `loo()` to diagnose problems**
3. **Make out-of-sample predictions and evaluate with proper scoring rules**

Don't interpret coefficients

```
coef(model)
```

##		2.5%	50%	97.5%	Rhat	n_eff
##	(Intercept)	0.4244452	0.9973625	1.87272150	1.01	377
##	s(season).1	-0.6285627	-0.3477210	-0.06202533	1.00	1661
##	s(season).2	-0.7338488	-0.3920205	-0.05727605	1.00	1993
##	s(season).3	-0.4865379	-0.1625075	0.13829287	1.00	2374
##	s(season).4	-0.1564071	0.1396100	0.42328822	1.00	2229
##	s(season).5	0.2798351	0.5486395	0.82546612	1.00	2148
##	s(season).6	0.1263498	0.3864325	0.63401010	1.00	2116

Don't interpret coefficients

These coefficients are acting on the *link scale*

Often result in nonlinear relationships on response scale

Very often, the coefficients are *correlated somehow*

This is especially the case in GAMs!

Don't worry about p -values or intervals, use *posterior predictions* instead

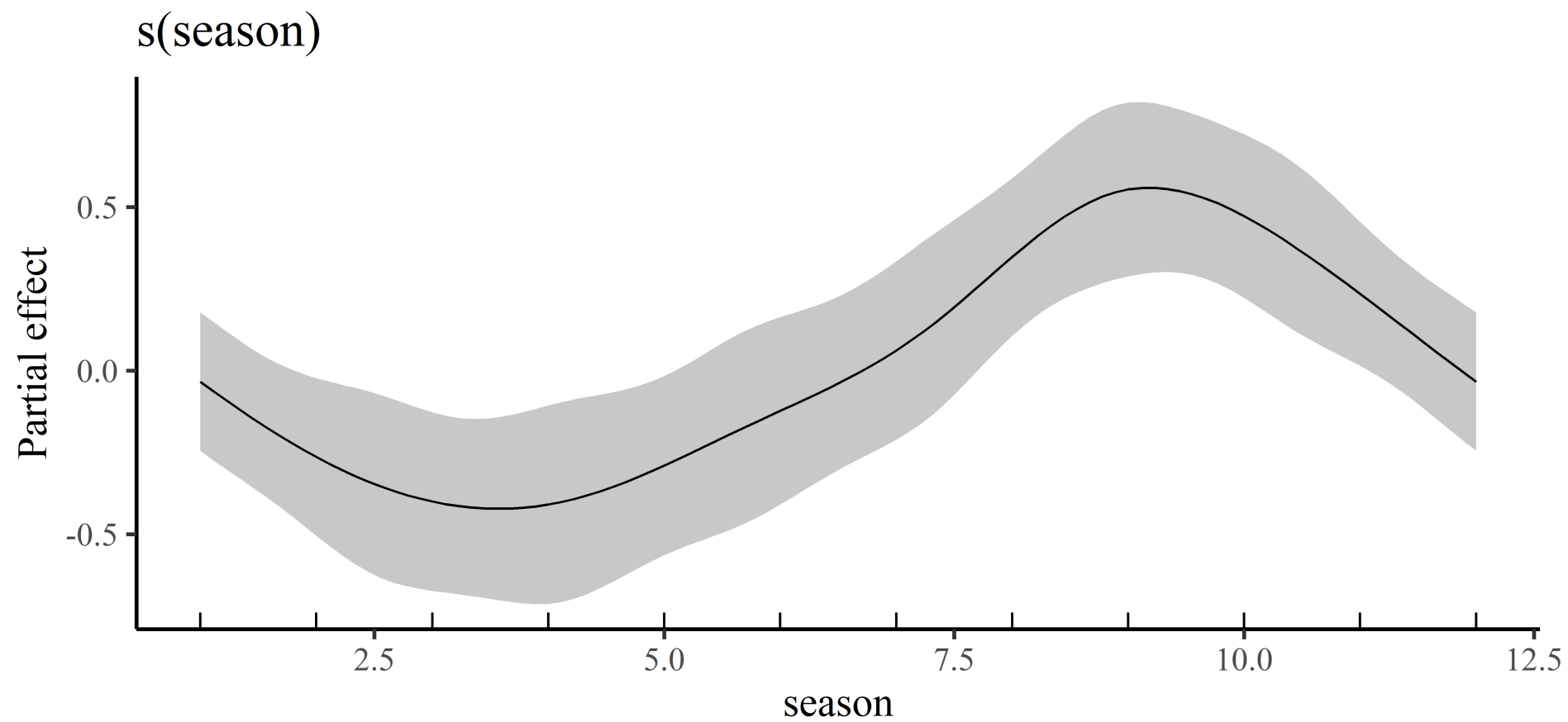
Start with *partial effects* on link scale

These are conditional on all other effects being zero

negative values $\Rightarrow$ covariate reduces the response

positive values $\Rightarrow$ covariate increases the response

`draw(model)`



Basis: Cyclic CRS

link predictions

```
predict(object, type = 'link')
```

Gives the real-valued, unconstrained linear predictor

Takes into account uncertainty in GAM regression coefficients

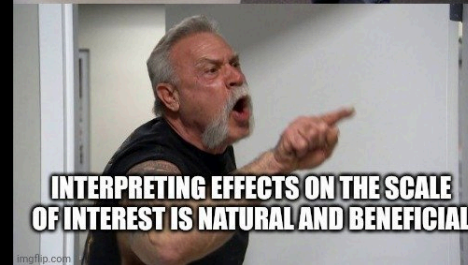
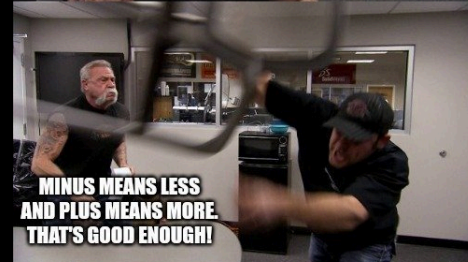
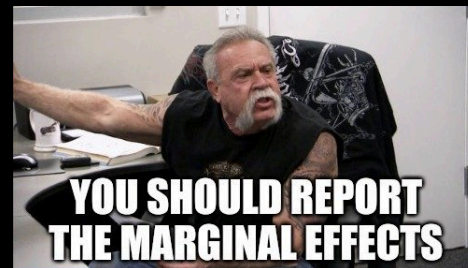
Can include uncertainty in any dynamic trend components

link predictions

Code Plot

```
# extract link-scale forecasts from the model  
fc ← forecast(model, type = 'link')  
  
# plot using the available S3 plotting function  
plot(fc)
```


Ok. but what do these things actually, really *mean*?



[Credit @stephenjwild](#)

expected predictions

```
predict(object, type = 'expected')
```

Gives the *average* prediction on the observation (response) scale

Useful as we often want to get a sense of long-term averages for guiding scenario analyses

Usually it is just the inverse link function applied to a prediction from `type = link`

This is the most confusing type of prediction, as using `predict(model, type = 'response')` for models fitted in `gam()` or `glm()` actually return *expectations*

expected predictions

Code Plot

```
# extract expectation-scale forecasts from the model  
fc ← forecast(model, type = 'expected')  
  
# plot using the available S3 plotting function  
plot(fc)
```

response predictions

```
predict(object, type = 'response')
```

Gives the predictions on the observation (response) scale

Includes uncertainty in the linear predictor *and* any uncertainty arising from the observation process

Some distributions only depend on the inverse link of the linear predictor (i.e. *Poisson*(λ) or *Bernoulli*(π))

Others depend on additional shape / scale parameters (i.e. *Normal*(μ, σ) or *StudentT*(ν, μ, σ))

These are the most often used for evaluating forecasts

Interpreting on the *response* scale

Some key questions you should ask of a fitted model

Can the model simulate realistic data?

Does the model capture salient features of the data that you'd like to predict?

What criteria would you use to determine whether one model is more suitable than another?

Very often, these questions can only be answered by looking at what kinds of predictions a model makes ***on the response scale***

response predictions

Code Plot

```
# extract response-scale forecasts from the model  
fc ← forecast(model, type = 'response')  
  
# plot using the available S3 plotting function  
plot(fc)
```

Posterior predictive checks

Posterior predictive checks

Statistical models can be used to generate (i.e. simulate) new outcome data

Can either use the same covariates used to train the model

Or can use `newdata` for scenario modelling (including forecasting)

To generate new outcome data we can simulate from the model's posterior predictive distribution

"The idea is simple: if a model is a good fit then we should be able to use it to generate data that looks a lot like the data we observed"

Gabry & Mahr

A PPC barplot

Code Plot

```
# view barplots of true data vs simulated predictions  
pp_check(model, type = 'bars', ndraws = 25)
```

A PPC cumulative distribution

Code Plot

```
# view the simulated vs true cumulative distribution functions  
pp_check(model, type = 'ecdf_overlay', ndraws = 25)
```

A PIT CDF

Code Plot

```
# view the simulated vs true count frequencies  
pp_check(model, type = 'pit_ecdf')
```

Comparing fits with `loo()`

```
model_good ← mvgam(  
  y ~ s(season, bs = 'cc', k = 8) +  
    gp(time, k = 20, c = 5/4, scale = FALSE),  
  data = data_train,  
  trend_model = 'None',  
  family = poisson()  
)
```

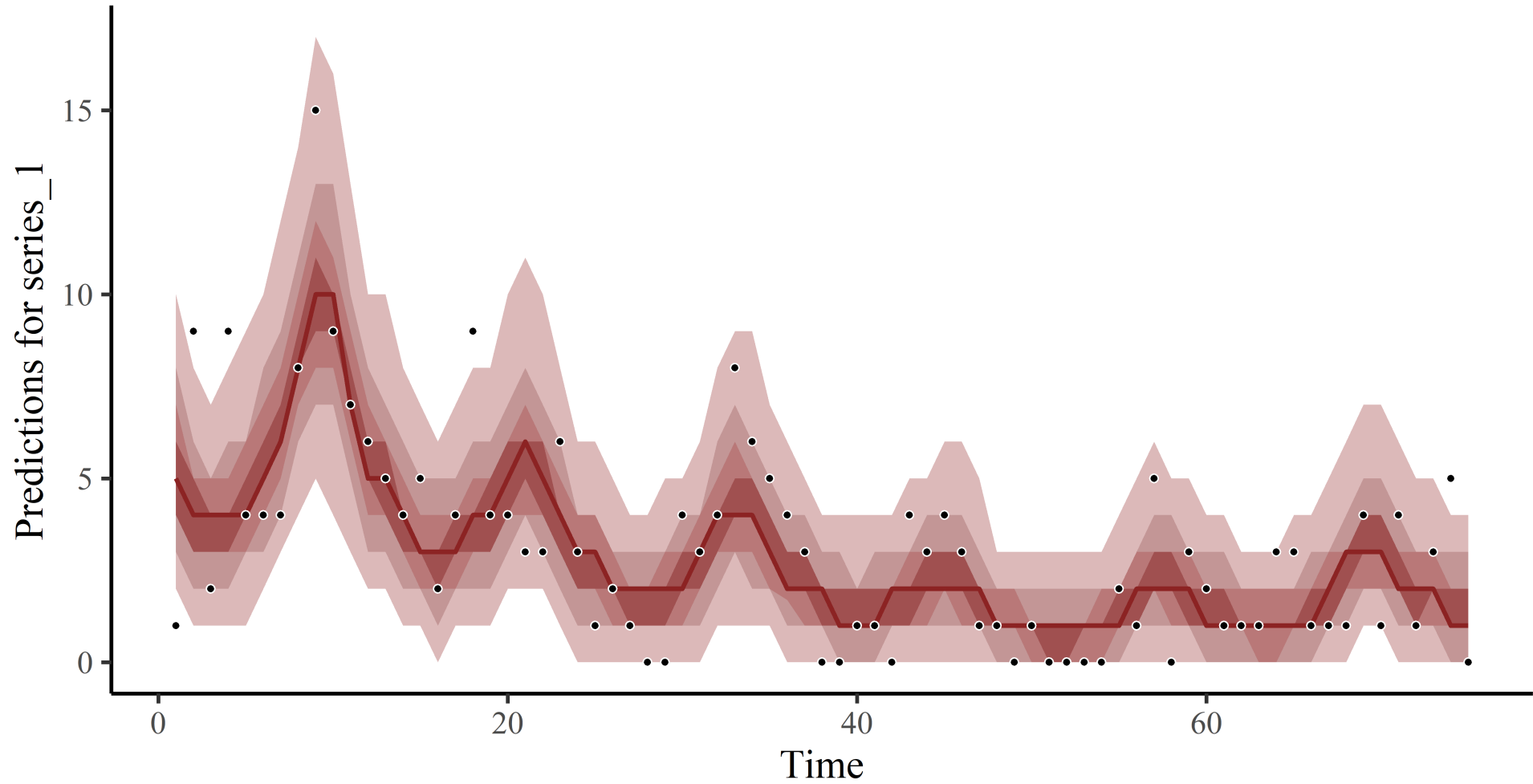
A GP of `time`, together with the cyclic seasonality, is a good model here

Comparing fits with `loo()`

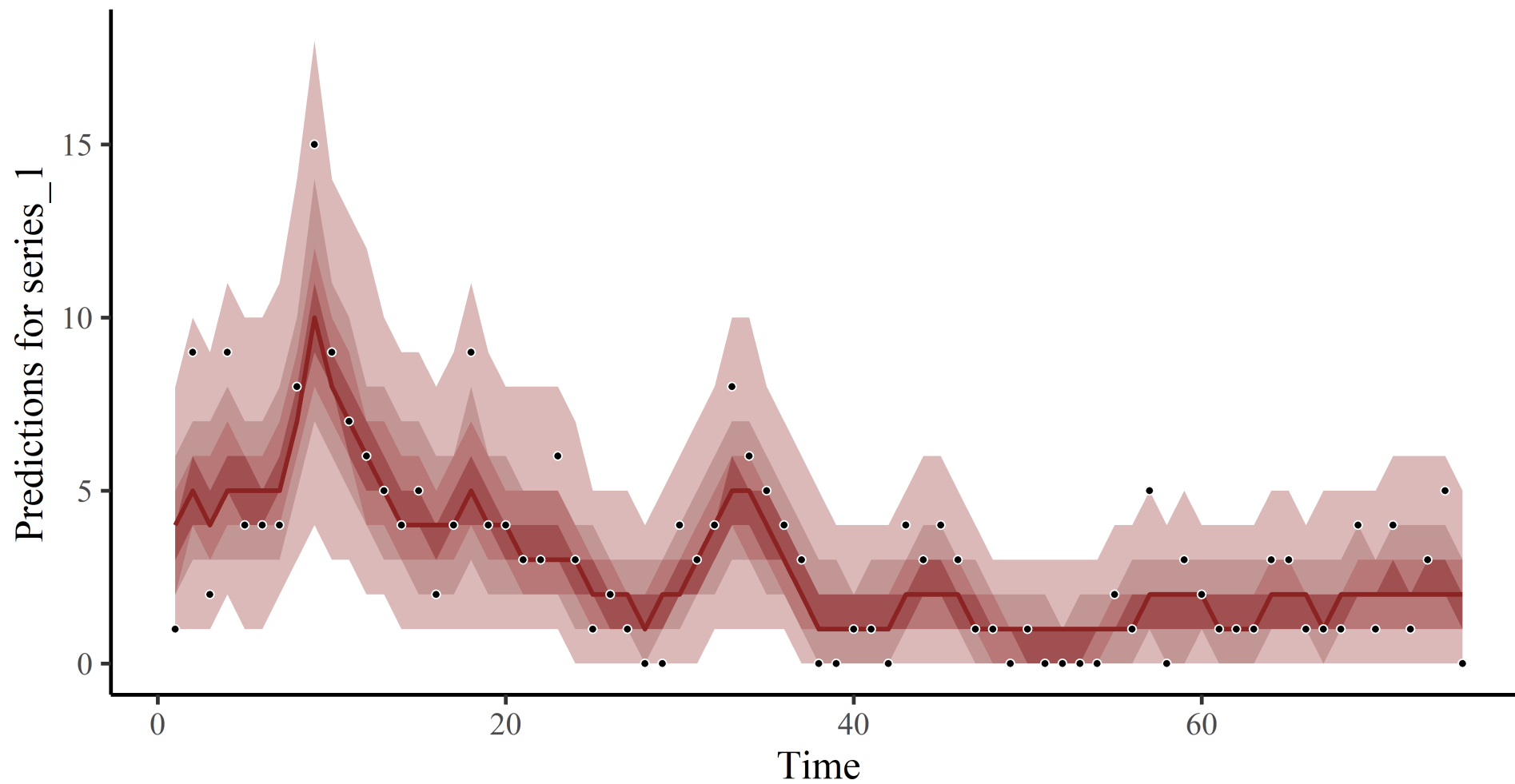
```
model_bad <- mvgam(  
  y ~ 1,  
  data = data_train,  
  trend_model = RW(),  
  family = poisson()  
)
```

A RW with no seasonality will fit the data *very well*, but will not *generalize* well

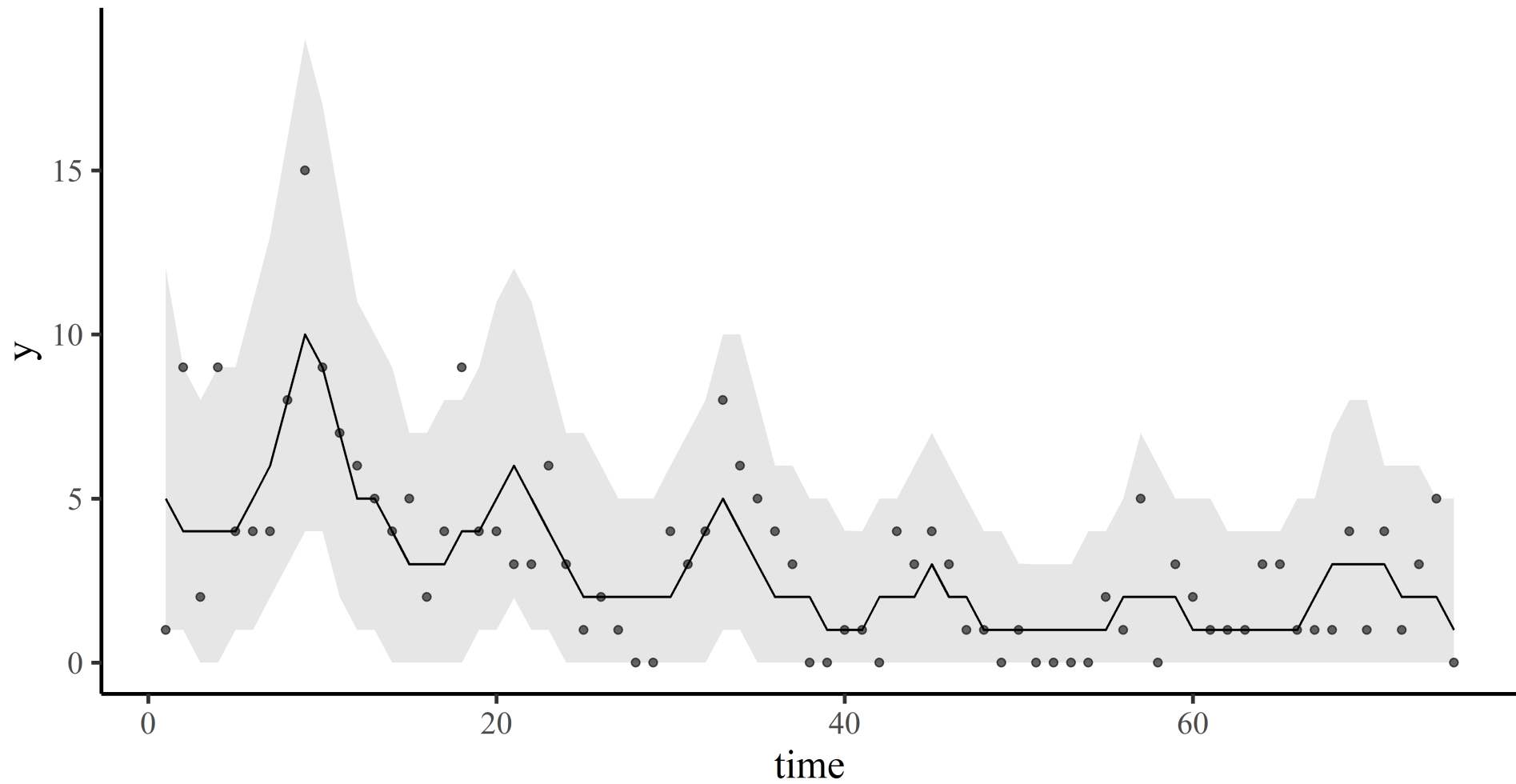
```
plot(hindcast(model_good))
```



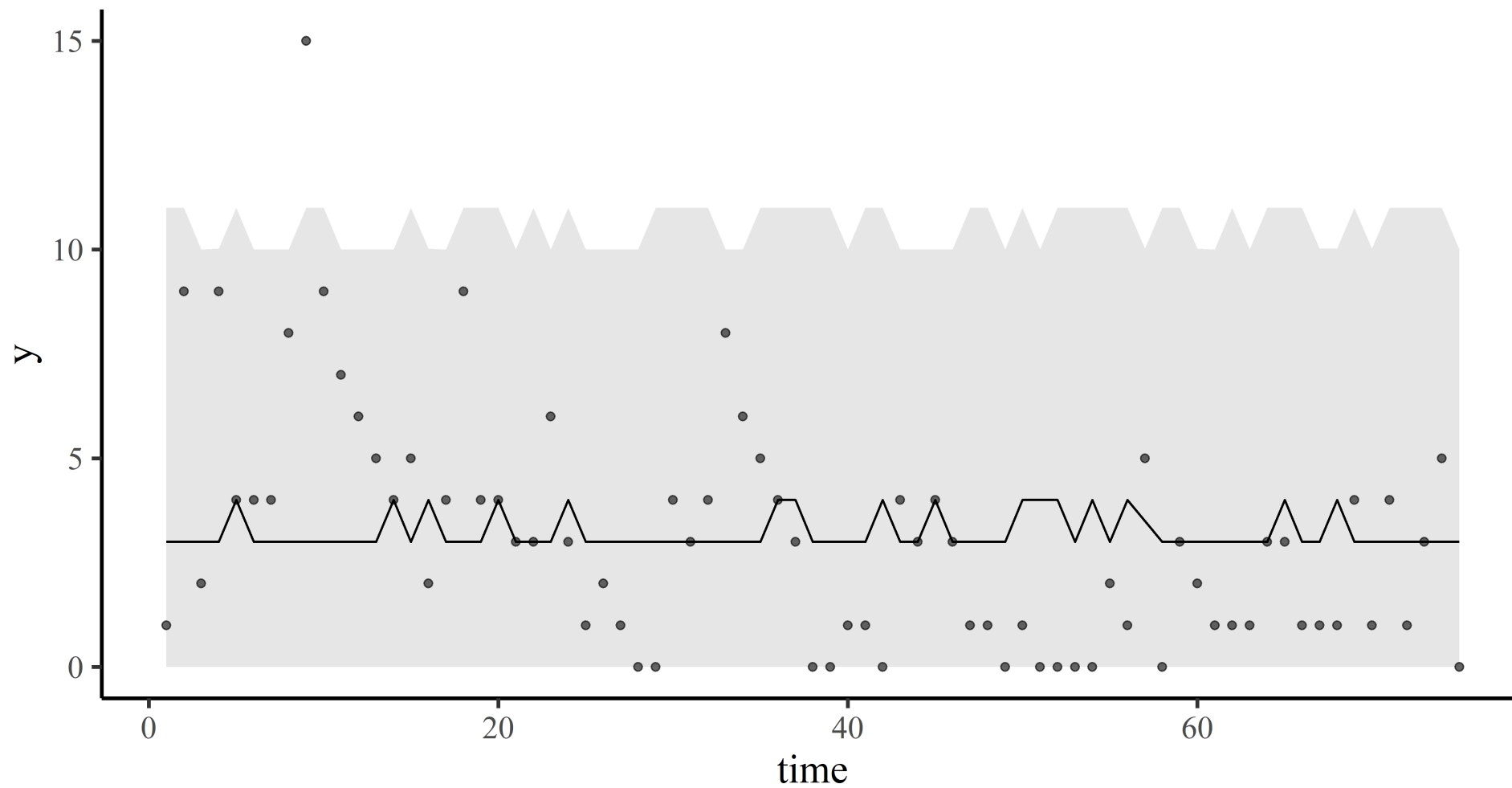
```
plot(hindcast(model_bad))
```




```
plot_predictions(model_good, by = 'time')
```



```
plot_predictions(model_bad, by = 'time')
```



Comparing fits with `loo()`

```
loo_compare(model_good, model_bad)
```

```
##           elpd_diff se_diff  
## model_good      0.0      0.0  
## model_bad    -300.4     33.8
```

A much lower Expected Log Predictive Density (ELPD) for `model_bad` suggests that this model would not do as well when tasked with predicting against a *new* set of observations

PPCs and `loo()` using training covariates are a great first step to check model validity and begin comparing models

But they only assess how well the model predicts against the training data

How else can we verify models? Using `newdata` for response predictions $\Rightarrow$ counterfactual *scenarios*

Prediction- based inferences

Marginal & conditional predictions

"Applied researchers are keen to report simple quantities that carry clear scientific meaning" ([Arel-Bundock 2023](#))

This is often challenging because:

- Intuitive estimands and uncertainties are tedious to compute

- Nonlinear terms, nonlinear link functions, interaction effects and observation parameters all make these effects nearly impossible to access from coefficients alone

- Most software emphasizes coefficients and p -values over meaningful interpretations

predict.mvgam()

Pass `newdata` consisting of particular covariate values that represent scenarios you'd like to explore

Can be simple: predict a smooth function along a fine-spaced grid to explore the smooth's shape and / or derivatives

Or can be complex: integrate over a high-dimensional grid of predictors to understand the average impact of a predictor on the response

Users can implement the wonderful `datagrid()` function from `marginalEffects`  to effortlessly generate a `data.frame` of covariate values for scenario predictions

Conditional smooths

Code Plot

```
# use plot_predictions() to visualise conditional effects  
# on the scale of the response  
plot_predictions(model,  
                  condition = 'season',  
                  points = 0.5,  
                  process_error = FALSE)
```


Rates of change

Code Plot

```
# use plot_slopes() to visualise rates of change  
# on the expectation scale of the response  
plot_slopes(model,  
             variables = 'season',  
             by = 'season',  
             type = 'expected',  
             process_error = FALSE) +  
geom_hline(yintercept = 0, linetype = 'dashed')
```

Finding peak growth

```
# estimate when the `season` smooth grows at its fastest rate
max_growth = function(hi, lo, x) {
  dydx ← (hi - lo) / 1e-6; x[dydx == max(dydx)][1]
}

comparisons(
  model, type = 'link',
  newdata = datagrid(season = seq(0, 12, length.out = 200)),
  variables = list("season" = 1e-6), comparison = max_growth
)
```

```
##
## Estimate 2.5 % 97.5 %
##      7.6  4.34  8.26
##
## Term: season
## Type: link
## Comparison: +1e-06
```

Posterior contrasts

Code Plot

```
# take draws of average comparison between season = 9 vs season = 3
avg_comparisons(
  model,
  variables = list(season = c(9, 3)),
  process_error = FALSE
) %>%
posterior_draws() %>%
# use the resulting posterior draw object to plot a density of the
# posterior contrasts
ggplot(aes(x = draw)) +
# use the stat_halfeye function from tidybayes for a nice visual
stat_halfeye(fill = "#C79999") +
labs(x = "(season = 9) - (season = 3)", y = "Density",
      title = "Average posterior contrast")
```

The ability to readily interpret models from `mvgam` and `brms`  's is a *huge advantage* over traditional time series models. See [my blogpost on interpreting GAMs for more examples](#)

But this is a forecasting course. So how can we evaluate forecast distributions?

The forecasting workflow

"The accuracy of forecasts can only be determined by considering how well a model performs on new data that were not used when fitting the model." Hyndman and Athanasopoulos

We must evaluate on data that was not used to train the model (i.e. ***leave-future-out cross-validation***) because:

Models that fit training data well do not always provide good forecasts

We can easily engineer a model that perfectly fits the training data, leading to overfitting

See the [mvgam forecasting vignette](#) for more guidance

Leave-future-out CV

Important to train the model on some portion of data and use a hold-out portion (test data) to evaluate forecasts:

$$p(y_{T+H} | y_{1:T})$$

Some points to consider:

- The test set should ideally be at least as large as the maximum forecast horizon required for decision-making

- Ideally, this process would be repeated many times to incorporate variation in forecast performance

- Usually good to compare models against simpler *benchmark* models to ensure added complexity improves forecasts

We must obtain leave-future-out forecasts (ideally for many different training / testing splits) to compare ecological forecasting models

But how do we *evaluate* forecasts?

The most common evaluation practice in forecasting tasks is to evaluate point predictions

Point-based forecast evaluation

Forecast errors

A forecast error (or forecast residual) is the difference between the true value in an out-of-sample set and the predicted response value:

$$\epsilon_{T+H} = \mathbf{y}_{T+H} - \hat{\mathbf{y}}_{T+H}$$

Where:

T is the total length of the training set

H is the forecast horizon

$\hat{\mathbf{y}}_{T+H}$ is the prediction at time $T + H$

Point-based measures use these errors in different ways

Common point-based measures

Scale-dependent measures

Mean Absolute Error: $mean(|\epsilon_t|)$

Root Mean Squared Error: $\sqrt{mean(\epsilon_t^2)}$

Scale-independent measures

Mean Absolute Percentage Error: $mean(|p_t|)$, where $p_t = 100\epsilon_t/y_t$

Mean Absolute Scaled Error: $mean(|q_t|)$, where q_t is the error scaled against errors from an appropriate **benchmark** forecast

Lower values are better for all these measures

We won't dwell much on point-based measures because ecological predictions and their associated management decisions are inherently *uncertain* (but see this video for more details)

Point-based measures ignore far too much information in the forecast distribution

It is better to evaluate the *entire forecast distribution*

Probabilistic forecast evaluation

Scaled Interval Score

A common step to evaluate a forecast distribution is to compute how well it's prediction intervals perform:

$$SIS = (U_t - L_t) + \frac{2}{\alpha}(L_t - y_t)\mathbf{1}(y_t < L_t) + \frac{2}{\alpha}(y_t - U_t)\mathbf{1}(y_t > U_t)$$

Where:

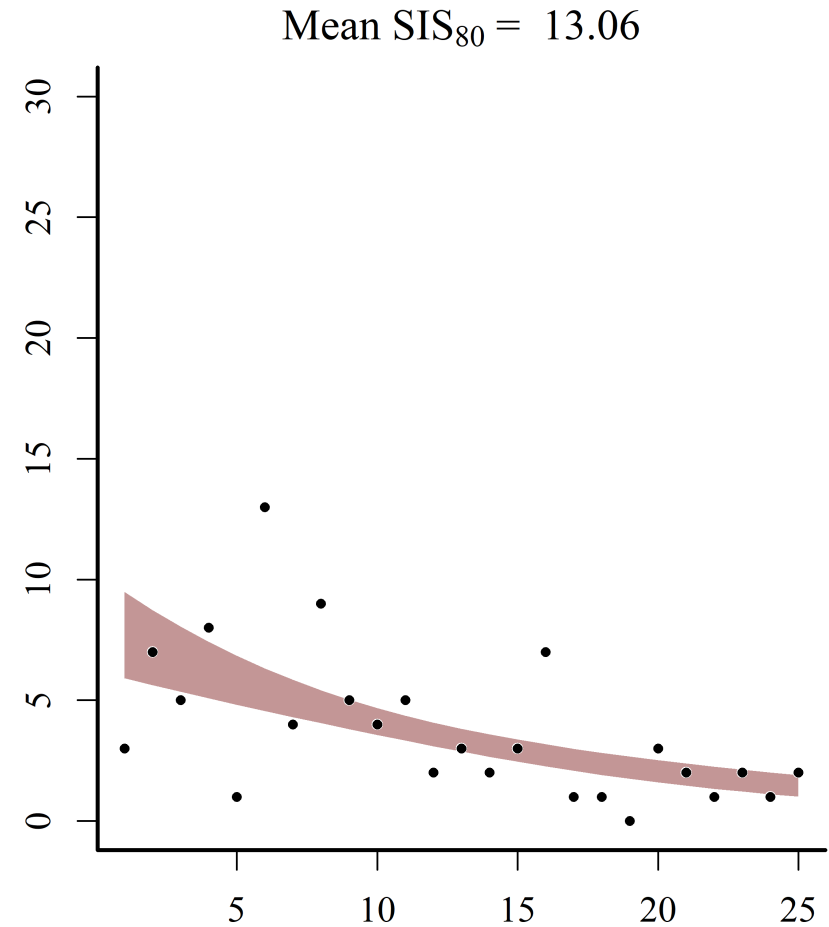
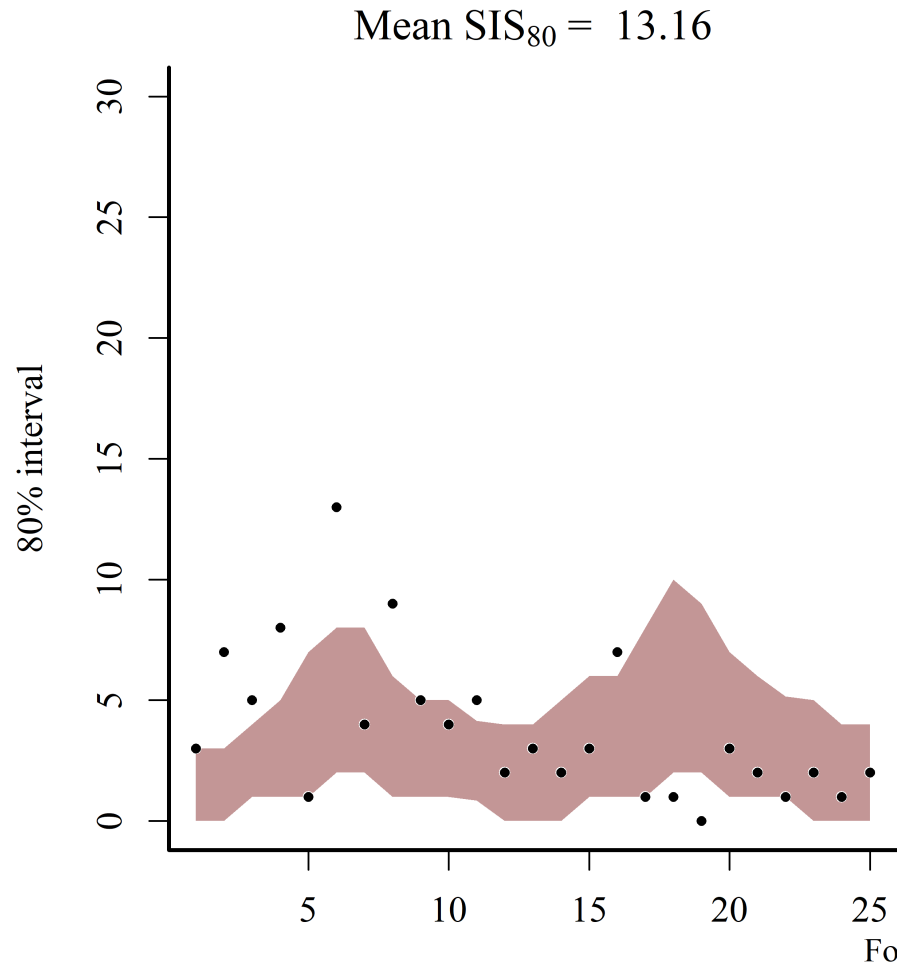
y_t is the true observed value at horizon H

α is 1 – interval width

The $100(1 - \alpha)\%$ interval for horizon H is $[L_t, U_t]$

$\mathbf{1}$ is a binary indicator function

Penalize *overly precise* forecasts



Evaluating the full distribution

Interval scores are very useful when we want to target a particular interval or if we don't have the full distribution

- Allows different teams to submit a few intervals rather than thousands of posterior samples

- Can compare forecasts from many different algorithms / models

But if we do have a full distribution, we have other options

"Scoring rules provide summary measures for the evaluation of probabilistic forecasts, by assigning a numerical score based on the predictive distribution and on the event or value that materializes"
([Gneiting and Raftery 2007](#))

What is a good forecast?

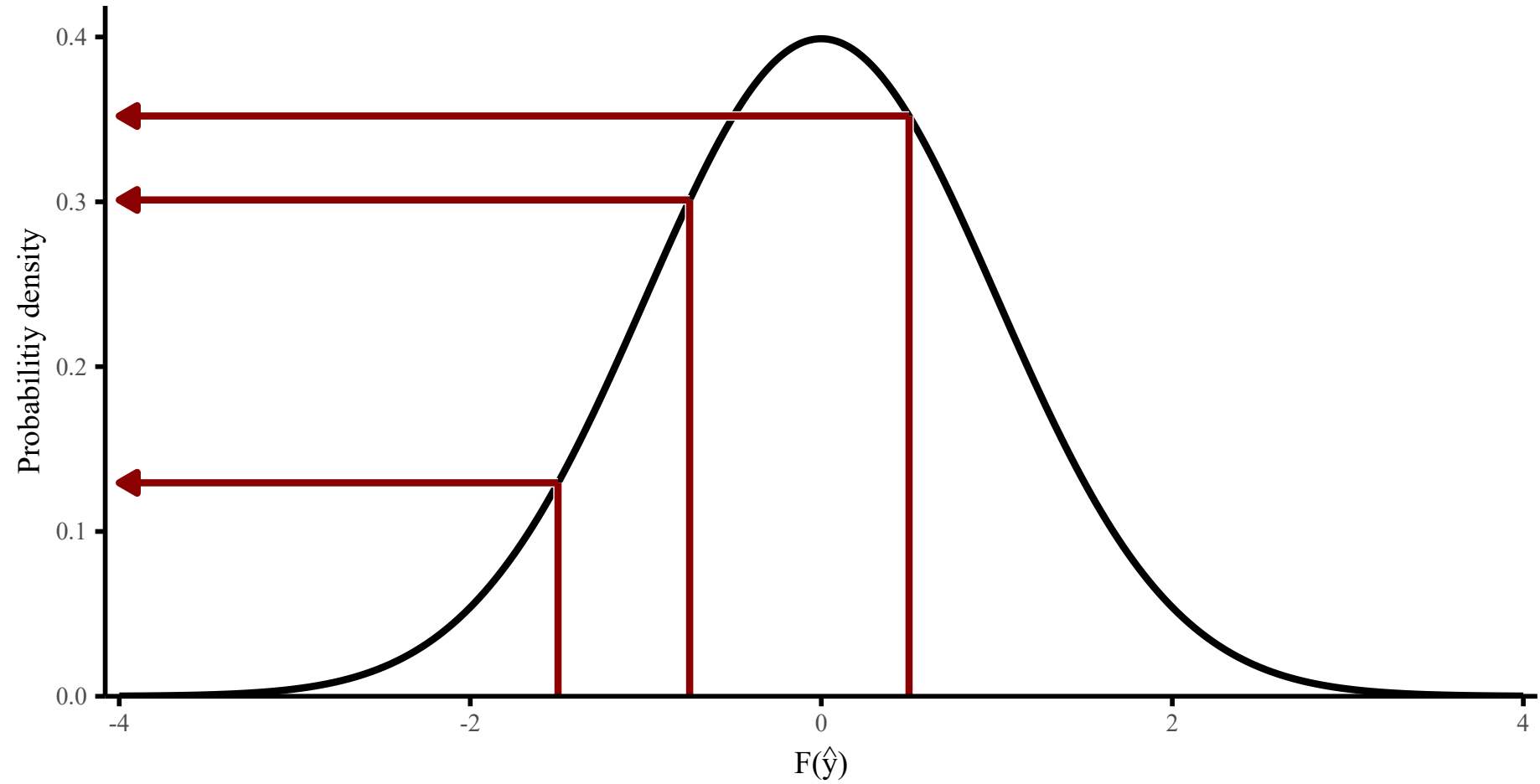
Reliable: good probabilistic calibration

Sharp: informative, with tight enough intervals to guide decisions

Skilled: performs better overall than simpler benchmark forecasts

Proper scoring rules attempt to address each of these goals using the full forecast distribution

Predictive density



Log predictive density

Compute $\log(\text{probability})$ of a given truth given distributional assumptions:

$$\log p(y_{T+H} | y_{t:T}, \theta)$$

Use density functions in , such as `dnorm` or `dnbinom`; higher values are better

θ captures all unknown parameters:

Regression coefficients β

Dynamic parameters; α or ρ for GP; σ_{error} for RW

Observation parameters; ν for StudentT or σ_{obs} for Normal

logging is stable and makes joint calculations easier

But the log score can severely penalize over-confidence and is sensitive to outliers

Other proper scoring rules can provide more robust comparisons, without needing to rely on distributional assumptions

CRPS

Continuous Ranked Probability Score compares true Cumulative Distribution Function (CDF) to forecast CDF

$$CRPS(F, y) = \int_{-\infty}^{\infty} (F(\hat{y}) - \mathbf{1}(\hat{y} \geq y))^2 dy$$

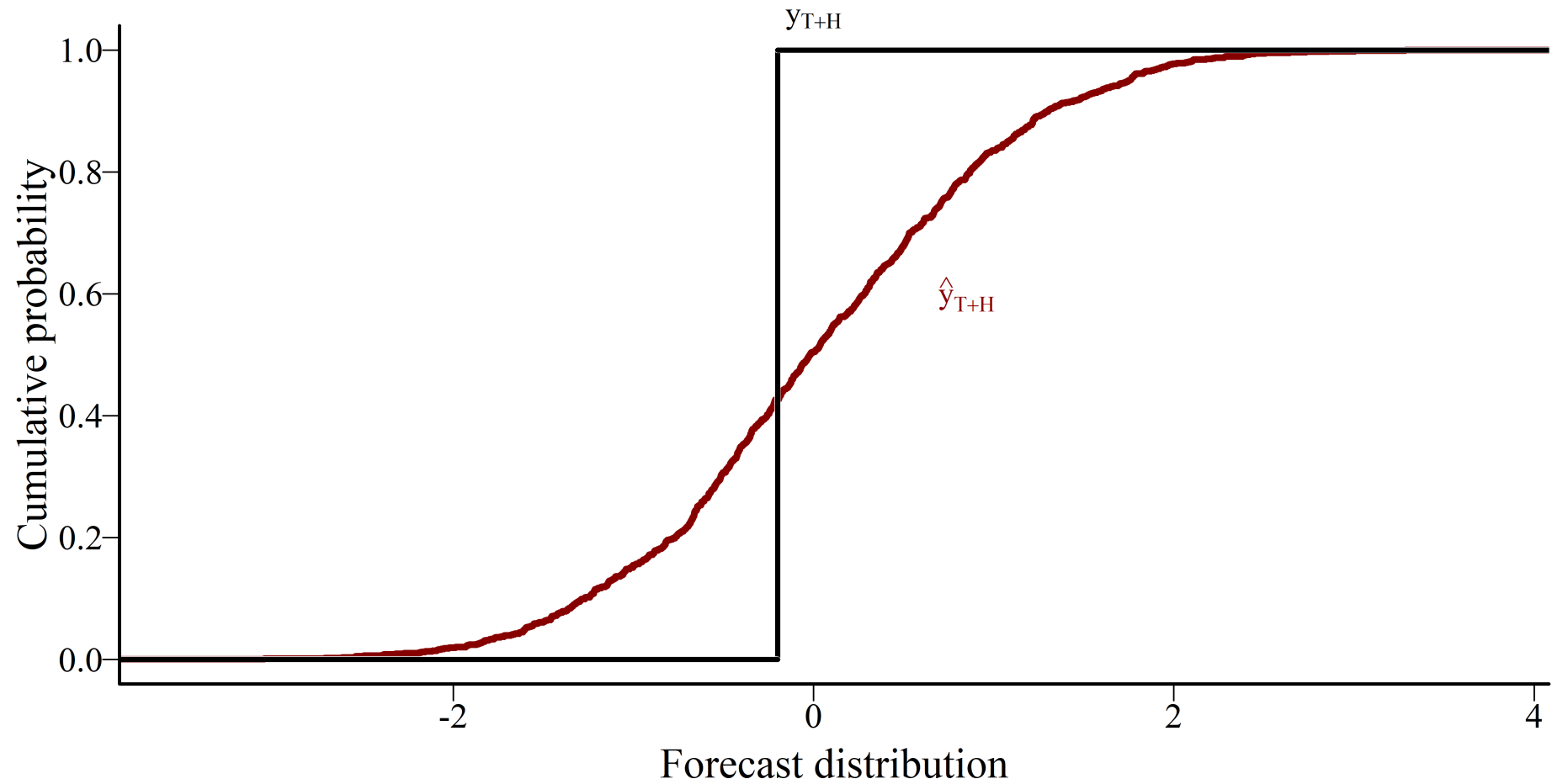
Where:

$F(\hat{y})$ is the forecast CDF evaluated at many points

$\mathbf{1}(\hat{y} \geq y)$ gives the true observed CDF

SIS converges to CRPS when evaluating an increasing number of equally spaced intervals

CRPS



**CRPS useful for both parametric and non-parametric predictions
because we just need to calculate the CDF of the forecast
distribution**

**Penalises over- and under-confidence similarly, and gives more
stable handling of outliers**

**Score is on the scale of the outcome variable being forecasted, so
is somewhat intuitive (a lower score is better)**

DRPS

Similar to CRPS, the discrete version (DRPS) can be used to evaluate a forecast that is composed only of integers

Uses an approximation of the forecast and true CDFs at a range of possible count values

Interpretation is similar

score.mvgam_forecast()

Once forecasts are computed and stored in an object of class `mvgam_forecast`, scores can be directly applied

User chooses among the Scaled Interval Score (`sis`), log score (`elpd`), CRPS (`crps`), DRPS (`drps`) and two multivariate scores (`energy` or `variogram`; more on this in the next lecture)

User also specifies an interval for calculating coverage and/or which interval to use for the Scaled Interval Score

`return` is a `list()` with scores for each series in the data and an overall score (usually just the sum of series-level scores)


```
sc ← score(forecast(model),  
           score = 'crps',  
           interval = 0.90)  
sc$series_1[1:10,]
```

##	score	in_interval	interval_width	eval_horizon	score_type
## 1	0.9475925	1	0.9	1	crps
## 2	4.2706693	0	0.9	2	crps
## 3	1.9558260	1	0.9	3	crps
## 4	4.0117590	0	0.9	4	crps
## 5	1.6035490	1	0.9	5	crps
## 6	6.3143580	0	0.9	6	crps
## 7	0.6937460	1	0.9	7	crps
## 8	3.9872183	1	0.9	8	crps
## 9	1.4215897	1	0.9	9	crps
## 10	0.8361572	1	0.9	10	crps

Calculating the CRPS using the previously generated forecasts

```
sc ← score(forecast(model),  
           score = 'sis',  
           interval = 0.90)  
sc$series_1[1:10,]
```

##	score	in_interval	interval_width	eval_horizon	score_type
## 1	4	1	0.9	1	sis
## 2	45	0	0.9	2	sis
## 3	6	1	0.9	3	sis
## 4	27	0	0.9	4	sis
## 5	8	1	0.9	5	sis
## 6	50	0	0.9	6	sis
## 7	10	1	0.9	7	sis
## 8	9	1	0.9	8	sis
## 9	8	1	0.9	9	sis
## 10	8	1	0.9	10	sis

Calculating the SIS using the previously generated forecasts; values outside interval are more heavily penalized

How does model_good perform?

Code Plot

```
fc_good ← forecast(  
  model_good,  
  newdata = data_test  
)  
plot(fc_good)
```

How does `model_bad` perform?

Code Plot

```
fc_bad <- forecast(  
  model_bad,  
  newdata = data_test  
)  
plot(fc_bad)
```

How would an *ensemble* perform?

Code Plot

```
ens ← ensemble(  
  fc_good,  
  fc_bad  
)  
plot(ens)
```

We have seen how to produce out-of-sample forecasts from `mvgam` models and evaluate them against new observations

We have also investigated other ways that models can be critiqued, particularly making use of conditional predictions using `newdata`

**But so far we have only considered univariate investigations.
What happens if we want to forecast *multiple time series*?**

In the next lecture, we will cover

Multivariate ecological time series

Vector autoregressive processes

Dynamic factor models

Multivariate forecast evaluation